

AMBRIES_ – TUTORIEL D'INSTALLATION

Ce tutoriel installe l'overlay dans **OBS Studio**, de zéro jusqu'au premier live, puis habille ta **chaîne Twitch**.
Chaque page a sa **fiche** : tu peux suivre une fiche seule, sans lire le reste.

💡 Pour tout voir en direct : ouvre `index.html` dans ton navigateur. La version à lire confortablement de ce tutoriel est `TUTO.pdf` .

SOMMAIRE

1. Ce qu'il y a dans le dossier
2. Avant de commencer (config, réglages OBS, les 3 gestes de base)
3. Les scènes, fiche par fiche : Démarrage · Pause · Fin · Cam seule · Contenu · Jeu
4. Les sources à la carte : Alertes · Chat · Bandeau · Objectif · Cadre cam
5. Les transitions : Splash · Gouttes · Pop art
6. Brancher le chat et les alertes (Streamer.bot, journal, objectif, sons des alertes)
7. Habiller la chaîne Twitch
8. Tester sans être en live
9. Personnaliser
10. Dépannage

1. CE QU'IL Y A DANS LE DOSSIER

```
ambries/
├─ reglages.html      ← LA page pour changer les textes et réglages (elle écrit mes-
reglages.js)
├─ config.js          ← les valeurs par défaut (remplacé à chaque mise à jour de
l'overlay)
├─ mes-reglages.js    ← TES réglages, écrits par reglages.html (apparaît au 1er
enregistrement ; à garder)
├─ index.html         ← aperçu de tout
├─ TUTO.md / TUTO.pdf ← ce tutoriel
├─ CONCEPT.md / .pdf ← le résumé du projet (DA, choix, ce qu'il reste à faire)
├─ scenes/           ← les écrans et overlays de scène (une page = une scène OBS)
├─ sources/          ← les éléments à poser où tu veux (alertes, chat...)
├─ sons/             ← tes propres sons d'alerte, si tu veux (facultatif, voir 6.8)
├─ transitions/      ← les transitions (+ videos/ : prêtes pour OBS)
├─ chaine/           ← kit de chaîne Twitch (kit.html + export/ : les PNG)
├─ outils/           ← les scripts (vidéos, images du kit, PDF) et actualiser-
obs.lua pour OBS : voir 9
├─ assets/avatar.png  ← ta photo de profil (utilisée partout)
├─ design/moodboard.html
└─ css/ js/          ← le moteur (pas besoin d'y toucher)
```

Toutes les pages font **1920 × 1080**, avec un fond transparent là où il faut.

2. AVANT DE COMMENCER

2.1 Remplir les réglages avec `reglages.html`

1. Dans le dossier de l'overlay, double-clique sur `reglages.html` : la page s'ouvre dans ton navigateur (**Edge** ou **Chrome**).
2. Vérifie au minimum, dans **La chaîne** : le **nom affiché** et ton **identifiant Twitch** (celui de l'adresse `twitch.tv/...`).
3. Dans **Objectif** : mets ton nombre **actuel** de followers dans **Ton nombre ACTUEL**.
4. Clique  **Enregistrer mes réglages** (en bas, ou **Ctrl + S**).
5. **La première fois**, une fenêtre s'ouvre : choisis le **dossier de l'overlay** (celui qui contient `reglages.html`), puis **Sélectionner le dossier**. Le navigateur demande s'il peut modifier les fichiers : clique **Modifier les fichiers** (ou **Autoriser**). La page écrit alors tes réglages dans le fichier `mes-reglages.js`, à côté de `config.js` : rien à copier à la main. Les fois suivantes, elle s'en souvient et enregistre directement (au plus, le navigateur redemande l'autorisation).
6. Dans OBS : **clic droit sur la source > Actualiser** pour voir le changement.

Un **point** à côté d'un réglage veut dire qu'il a changé et n'est pas encore enregistré. La section **Alertes** montre un aperçu de chaque alerte avec tes textes, et la section **Tester** ouvre les pages en mode test.

Où vont tes réglages ? `config.js` contient les **valeurs par défaut** de l'overlay ; `mes-reglages.js` contient **seulement ce que tu as changé**, appliqué par-dessus. En haut de la page, un encadré dit combien de réglages perso tu as. Remettre un réglage à sa valeur d'origine le retire de `mes-reglages.js`.

 Avec **Firefox**, la page ne peut pas écrire dans le dossier : elle **télécharge** `mes-reglages.js` , à mettre dans le dossier de l'overlay (à la place de l'ancien). Préfère Edge ou Chrome.

À la main (sans la page) : ouvre `config.js` avec le **Bloc-notes** (clic droit > *Ouvrir avec* > *Bloc-notes*). Garde les guillemets `"..."` autour des textes et la virgule `,` en fin de ligne, enregistre, puis **Actualiser** dans OBS. Attention : un changement fait à la main dans `config.js` sera perdu à la prochaine mise à jour ; dans `reglages.html` , il est gardé.

Quand tu reçois une nouvelle version de l'overlay

1. Copie les nouveaux fichiers par-dessus les anciens (remplacer).
2. Ton `mes-reglages.js` **n'est pas dans la nouvelle version** : il reste en place, tes réglages sont gardés. Les nouvelles fonctions arrivent avec leurs valeurs par défaut.
3. Dans OBS : actualise les sources.

 **La toute première fois seulement** (si tes réglages étaient encore dans l'ancien `config.js`) : **avant** de remplacer les fichiers, fais une copie de ton `config.js` (ex. sur le bureau). Après la mise à jour, ouvre `reglages.html` >  **Reprendre les réglages d'un ancien config.js** > choisis cette copie : tes réglages reviennent dans le formulaire (marqués `-`). Clique **Enregistrer**, c'est fini : ils sont maintenant dans `mes-reglages.js` .

2.2 Régler le canevas d'OBS (une seule fois)

Paramètres > Vidéo : résolution de base `1920 × 1080` , résolution de sortie `1920 × 1080` .
Les positions des fiches sont données pour ce canevas.

2.3 Geste A – ajouter une source Navigateur

C'est la même manipulation pour **tous** les fichiers `.html` :

1. Sélectionne la scène, puis dans **Sources : + > Navigateur**.
2. Donne un nom à la source, puis **OK**.
3. Coche **Fichier local, Parcourir**, choisis le fichier `.html` indiqué dans la fiche.
4. **Largeur** : `1920` , **Hauteur** : `1080` .
5. Coche les cases indiquées dans la fiche.
6. **OK**.

2.4 Geste B – placer une source au pixel près

1. Clique sur la source (webcam, jeu, capture), puis **Ctrl + E**.
2. **Position** : x et y de la fiche.
3. **Type de zone de délimitation** : *Mettre à l'échelle à l'extérieur de la zone*.
4. **Taille de la zone de délimitation** : largeur et hauteur de la fiche.
5. Coche **Rogner à la zone de délimitation**, puis **Fermer**.

💡 **Voir les chiffres directement dans OBS :** `reglages.html` > **Options des scènes** > coche **Afficher la taille et la position des zones**, puis **Enregistrer** et actualise. Chaque zone (webcam, contenu, jeu) affiche sa taille et sa position : tu les recopies dans Ctrl + E. Décoche ensuite. (Ou, pour une seule source : `?zones=1` dans l'adresse, geste C.)

Les zones de la cam ont des **coins arrondis** : l'overlay recouvre les angles de ta webcam, c'est normal.

2.5 Geste C – ajouter une option dans l'adresse

1. Dans la source Navigateur, **décoche** *Fichier local*.
2. Colle l'adresse complète dans **URL**, avec les options après un `?` :

```
file:///G:/Projets/Overlay/ambries/scenes/demarrage.html?minutes=10
```

Plusieurs options se séparent par `&` : `...jeu.html?cam=bas-gauche&test=1`

Plus simple, et pour de bon : `reglages.html` > **Options des scènes**. Le coin de la webcam de la scène Jeu (ou « Pas de webcam »), le chat, le bandeau de chaque scène s'y règlent une fois pour toutes, sans toucher aux adresses dans OBS. Une option écrite dans l'adresse d'une source passe **avant** ce réglage (pratique pour une source particulière).

2.6 Astuce – une seule source d'alertes pour toutes les scènes

1. **Scènes** > + : crée une scène « **Global – Alertes** » et ajoutes-y `sources/alertes.html` (fiche 4.1).
2. Dans chaque autre scène : **Sources** > + > **Scène** > « Global – Alertes », tout **en haut** de la liste.

🔑 **Règle d'or** : dans OBS, ce qui est **en haut** de la liste s'affiche **par-dessus**.

2.7 Le chat et le bandeau : déjà dans les scènes

Les scènes contiennent **déjà** leur chat et leur bandeau, placés pile à côté des zones de la cam et du jeu : **tu n'as rien à ajouter**.

La source séparée `sources/chat.html` (ou `sources/bandeau.html`) sert seulement pour une scène à toi, ou pour placer le chat autrement : dans ce cas, ajoute `?chat=0` (ou `?bandeau=0`) à l'adresse de la scène pour retirer celui qui est intégré, puis ajoute la source séparée.

Quand tu changes de scène, le chat **réaffiche les derniers messages** (ceux des 10 dernières minutes, réglable dans `reglages.html` > **Le chat**) : il ne repart pas à vide.

2.8 Actualiser toutes les sources d'un coup (après un changement de réglages)

OBS n'a pas de bouton pour actualiser toutes les sources Navigateur : l'overlay en fournit un, sous forme de petit script OBS.

1. OBS > **Outils** > **Scripts** > onglet **Scripts** > + > choisis `outils/actualiser-obs.lua` (dans le dossier `ambries`).
2. À droite : le bouton **Actualiser toutes les sources Navigateur**, et la case **Actualiser tout seul les sources de l'overlay quand les réglages changent** (cochée) : après **Enregistrer** dans `reglages.html`, les sources se mettent à jour en 2 secondes.

3. **Fermer** : le script reste installé. Raccourci clavier possible : **Paramètres > Raccourcis clavier > « Actualiser toutes les sources Navigateur »**.

⚠ Une page actualisée repart de zéro (un compte à rebours recommence).

Placer les webcams tout seul (le même script)

Dans la même fenêtre **Outils > Scripts**, le script a aussi :

- le bouton **Placer les webcams sur toutes les scènes (et les ajouter là où elles manquent)** : dans chaque scène qui affiche une page de l'overlay (`scenes/jeu.html` , `contenu.html` , `cam-seule.html` ...), ta webcam est mise **pile dans sa zone**, à la bonne taille, et rognée. Si une scène n'a pas encore de webcam, elle y est ajoutée, juste sous l'overlay ;
- la case **Replacer tout seul les webcams quand les réglages changent** (cochée) : tu changes la webcam de la scène Jeu dans `reglages.html` > **Options des scènes**, tu enregistres... et dans OBS la webcam se déplace en même temps que son cadre.

Pour que le script reconnaisse ta webcam, **son nom doit contenir « cam »** (ex. « Webcam »). Une carte d'acquisition de console n'est jamais déplacée (sauf si son nom contient « cam »). Une scène réglée **Pas de webcam** : la webcam y est cachée, puis réaffichée quand tu la remets. Les positions viennent de `js/zones.js` (les mêmes que dans les fiches ci-dessous) ; si ton canevas est en 1440p, le script fait la conversion tout seul.

Le rognage automatique demande **OBS 30.1 ou plus récent**. Avec un OBS plus ancien, si la webcam déborde de son cadre : clic droit > **Transformer > Rogner** à la main (geste B).

3. LES SCÈNES, FICHE PAR FICHE

3.1 🕒 Démarrage — `scenes/demarrage.html`

À quoi ça sert : « Ça commence bientôt ! » avec ton avatar, le compte à rebours, la jauge « **Chargement des excuses...** » qui monte... et bloque à 90 %, et de petites phrases qui défilent. À zéro : « 90 %... ça suffira » et l'éclat « **C'est parti... (courage)** ».

Dans OBS :

1. **Scènes > +** : « Démarrage ».
2. **+ > Scène** > « Global — Alertes ».
3. Geste A avec `scenes/demarrage.html` , case **Actualiser le navigateur quand la scène devient active** cochée.
4. Ordre final : `Global — Alertes` · `Démarrage` .

Options (geste C) : `?minutes=10` (durée, 5 min par défaut) · `?heure=20:30` (heure fixe : le compteur arrive à zéro à 20 h 30 ; aussi dans `reglages.html` > ... **ou heure fixe**, prioritaire sur les minutes). Les phrases se changent dans `config.js` > `demarrage.phrases` .

Vérifier : ouvre `scenes/demarrage.html?minutes=0.5` dans ton navigateur, la fin arrive en 30 s.

3.2 🎬 Pause — `scenes/pause.html`

À quoi ça sert : l'enseigne néon « **Pause** » qui grésille, « Je reviens... avec une excuse. », le temps de pause et le chat qui reste visible.

Dans OBS :

1. **Scènes** > + : « Pause ».
2. + > **Scène** > « Global — Alertes ».
3. Geste A avec `scenes/pause.html` , case **Actualiser le navigateur quand la scène devient active** cochée (le chrono de pause repart de zéro).
4. Ordre final : `Global — Alertes` · `Pause` .

3.3 🎉 Fin — `scenes/fin.html`

À quoi ça sert : « **GG ! (enfin... presque)** », « Merci d'avoir assisté au carnage », ton dernier témoin et ton dernier soutien moral, sous une pluie d'éclaboussures.

Dans OBS :

1. **Scènes** > + : « Fin ».
2. Geste A avec `scenes/fin.html` , case **Actualiser le navigateur quand la scène devient active** cochée.

Nécessite Streamer.bot pour afficher les derniers noms (section 6) ; sinon un tiret s'affiche.

3.4 📹 Cam seule — `scenes/cam-seule.html`

À quoi ça sert : la discussion : ta cam en grand dans son tube néon, le chat en bulles à côté, le bandeau en bas.

Dans OBS :

1. **Scènes** > + : « Cam seule ».
2. + > **Scène** > « Global — Alertes ».
3. Geste A avec `scenes/cam-seule.html` .
4. + > **Périphérique de capture vidéo** > « Webcam », choisis ta caméra.
5. Geste B sur la webcam : position `80` , `200` · taille `1260` × `709` .
6. Ordre final : `Global — Alertes` · `Overlay` · `Webcam` .

Vérifier : adresse `cam-seule.html?apercu=1` (geste C), la zone de la cam est grisée et légendée. Retire `apercu=1` ensuite.

3.5 📺 Contenu — `scenes/contenu.html`

À quoi ça sert : la cam et le chat en colonne à gauche, le contenu (jeu, vidéo, navigateur...) en grand à droite.

Dans OBS :

1. **Scènes** > + : « Contenu ».
2. + > **Scène** > « Global — Alertes ».
3. Geste A avec `scenes/contenu.html` .
4. + > **Périphérique de capture vidéo** > « Webcam ».

5. + > **Capture de fenêtre** (ou *Capture d'écran, Capture de jeu, Navigateur...*) > « Contenu ».

6. Geste B :

Source	Position (x, y)	Taille (l x h)
Webcam	60 , 220	480 x 270
Contenu	590 , 200	1270 x 714

7. Ordre final : Global – Alertes · Overlay · Webcam · Contenu .

Vérifier : contenu.html?aperçu=1 (geste C).

Options : ?cam=0 (pas de webcam : le chat prend toute la hauteur ; n'ajoute pas la source Webcam).

3.6 🎮 Jeu – scenes/jeu.html

À quoi ça sert : le jeu en plein écran, une petite cam néon dans un coin, le chat en bulles qui s'effacent toutes seules après 45 secondes, un bandeau discret.

Dans OBS :

1. **Scènes** > + : « Jeu ».

2. + > **Scène** > « Global – Alertes ».

3. Geste A avec scenes/jeu.html .

4. + > **Périphérique de capture vidéo** > « Webcam ».

5. + > **Capture de jeu** > « Jeu ».

6. Geste B : le **Jeu** en 0 , 0 , 1920 x 1080 ; la **Webcam** en 400 x 225 , selon le coin :

Coin (option cam)	Position de la webcam (x, y)
bas-droite (défaut)	1456 , 735
bas-gauche	64 , 735
haut-droite	1456 , 190
haut-gauche	64 , 190

7. Ordre final : Global – Alertes · Overlay · Webcam · Jeu .

Options : ?cam=bas-gauche (le chat passe automatiquement de l'autre côté) · ?cam=0 (pas de webcam : pas de cadre, n'ajoute pas la source Webcam ; le chat reste à sa place) · ?cam=1200,700,420,236 (position perso : x, y, largeur, hauteur).

Position perso (au pixel près) : reglages.html > **Options des scènes** > **La webcam de la scène Jeu** > **Position perso** (X, Y, largeur, hauteur, ou fais glisser la cam sur le plan) ; ou dans l'adresse : ?

cam=1200,700,420,236 . **Le plus simple pour la poser dans OBS** : le bouton **Placer les webcams** du script actualiser-obs.lua (voir 2.8), qui suit aussi la position perso.

4. LES SOURCES À LA CARTE

À poser **en plus**, dans n'importe quelle scène. Position et taille avec `?x=&y=&l=&h=` (pixels 1920 × 1080, geste C).

4.1 🔔 Alertes — `sources/alertes.html`

À quoi ça sert : follows, abonnements, bits, raids et dons, en **sticker BD qui éclabousse**, avec un petit « pop » sonore. Les grands événements (raid, pluie d'abonnements, objectif atteint) ont en plus un éclat BD géant.

Dans OBS :

1. Dans la scène « Global — Alertes » (voir 2.6) : geste A avec `sources/alertes.html`.
2. Coche **Contrôler l'audio via OBS** et règle le volume dans le mélangeur audio.

Options : `?position=haut` (défaut), `centre` ou `bas` · `?test=1`

Nécessite Streamer.bot (section 6).

4.2 💬 Chat — `sources/chat.html`

À quoi ça sert : le chat en bulles BD seul, pour une scène à toi.

Dans OBS : geste A, puis geste C pour le placer.

Options : `?x=1400&y=180&l=460&h=760` · `?flottant=1` (bulles sans panneau) · `?disparition=45` · `?test=1`

4.3 📄 Bandeau — `sources/bandeau.html`

À quoi ça sert : ton avatar, le dernier témoin, le dernier soutien moral, les fonds pour excuses et l'objectif.

Dans OBS : geste A, puis geste C pour le placer.

Options : `?x=80&y=970&l=1760&h=84` · `?compact=1` (dernier témoin + objectif) · `?test=1`

Nécessite Streamer.bot (section 6).

Choisir les cases : `reglages.html` > **Bandeau d'infos** (par exemple, décoche le dernier soutien si tu ne reçois ni dons ni bits, et le dernier abonné si ta chaîne n'est pas encore affiliée).

4.4 ★ Objectif — `sources/objectif.html`

À quoi ça sert : la jauge où **ta tête avance** à chaque follow (ou abonnement, selon `config.js` > `objectif.type`).

Dans OBS : geste A, puis geste C pour le placer.

Options : `?x=560&y=60&l=800&h=150` · `?test=1`

Nécessite Streamer.bot (section 6).

4.5 📹 Cadre cam — `sources/cam.html`

À quoi ça sert : le tube néon seul (sticker « Ambries_ », « !! » qui tremble), pour une cam placée où tu veux.

Dans OBS :

1. Place ta webcam et note sa position et sa taille (Ctrl + E).
2. Geste A avec `sources/cam.html` **au-dessus** de la webcam, puis geste C avec les mêmes chiffres : `?x=1456&y=735&l=400&h=225`.

Options : `?nom=0` (sans plaque de nom) · `?titre=Coucou` (petit sticker au-dessus) · `?apercu=1`

5. LES TRANSITIONS

Les vidéos sont **déjà prêtes** dans `transitions/videos/` (fond transparent). Pour chacune :

1. Panneau **Transitions de scène** > + > **Stinger**, donne-lui un nom.
2. **Fichier vidéo** : la vidéo du tableau.
3. **Type de point de transition** : *Temps (millisecondes)* ; **Point de transition** : la valeur du tableau.
4. **OK**.

Transition	Vidéo	Point de transition
 Splash (bande de fluide blanc qui traverse et éclabousse)	<code>splash.webm</code>	1050 ms
 Gouttes (la peinture coule du haut puis le rideau tombe)	<code>gouttes.webm</code>	1150 ms
 Pop art (onde de choc, trame de points, éclat BD)	<code>pop-art.webm</code>	750 ms

Au milieu de chaque transition, ton avatar apparaît dans son anneau néon avec « Changement de scène ! ».

Une transition par scène : clic droit sur la scène > **Remplacer la transition**.

Sans vidéo : ajoute `transitions/splash.html` (ou une autre) tout en haut de la scène, case *Actualiser...* cochée, transition d'OBS sur **Coupure** : la scène apparaît en se découvrant.

Refaire les vidéos (après un changement de couleurs ou d'avatar), dans PowerShell :

```
cd G:\Projets\Overlay\ambries
node outils/generer-transitions.mjs
```

6. BRANCHER LE CHAT ET LES ALERTES

Deux choses différentes :

Quoi	Comment ça arrive	À installer
Le chat	l'overlay lit ton chat Twitch directement	rien : ton identifiant Twitch dans les réglages suffit
Les alertes, le bandeau, l'objectif et les derniers noms de l'écran de fin (follows, abonnements, bits, raids, dons)	par Streamer.bot , un logiciel gratuit qui tourne sur ton PC	Streamer.bot, une fois (6.2)

6.1 Le chat

1. `reglages.html` > **La chaîne** > **Identifiant Twitch** : celui de ton adresse `twitch.tv/...` . Enregistre.
2. C'est tout : dans les scènes, le chat s'affiche dès qu'un message arrive.

Les **emotes Twitch** s'affichent en image ; les **bots** (liste dans `reglages.html` > **Le chat**) et les **commandes** qui commencent par **!** sont cachés ; un message **supprimé par un modo**, ou ceux d'un spectateur **banni**, disparaissent aussi de l'overlay ; en changeant de scène, les messages des 10 dernières minutes sont réaffichés.

Il ne montre pas les messages envoyés **avant** l'ouverture d'OBS, ni les emotes des extensions 7TV, BTTV ou FFZ.

6.2 Installer Streamer.bot (une seule fois)

1. Va sur le site officiel **streamer.bot**, télécharge la dernière version (un fichier `.zip`).
2. Décompresse-le dans un dossier à toi (ex. `C:\Streamer.bot\`), puis lance `Streamer.bot.exe`.
3. **Connecter ta chaîne** : onglet **Platforms** > **Twitch** > **Accounts**. Dans la partie **Broadcaster** (ton compte de streamer), clique **Connect**, connecte-toi à Twitch dans la fenêtre qui s'ouvre, puis **Autoriser**. (La partie « *Bot* » est facultative.)
4. **Ouvrir la porte à l'overlay** : onglet **Servers/Clients** > **WebSocket Server** : **Address** `127.0.0.1` · **Port** `8080` · **Endpoint** / · coche **Auto Start** · clique **Start Server**.
5. Dans OBS, **clic droit sur la source d'alertes** > **Actualiser**.

✓ **À chaque live**, Streamer.bot doit être **lancé**. Avant ou après OBS, peu importe : l'overlay s'y reconnecte tout seul dès qu'il est là.

Si tu as changé le port ou mis un mot de passe dans Streamer.bot, reporte-les dans `reglages.html` > **Streamer.bot**.

6.3 Vérifier que l'overlay est bien branché

OBS n'a pas de console (F12) : l'overlay a donc son propre **journal**, affiché directement dans la source.

1. Dans OBS, double-clic sur la source d'alertes (dans la scène « Global – Alertes »).
2. Décoche **Fichier local** et colle dans **URL** l'adresse de la page suivie de `?journal=1` (geste C) :
`file:///G:/Projets/Overlay/ambries/sources/alertes.html?journal=1`
3. **OK** : un panneau sombre apparaît en haut à gauche. Tu dois y lire ✓ **Connecté à Streamer.bot**.
 - « ⚠ **Déconnecté de Streamer.bot** » : Streamer.bot n'est pas lancé, ou son serveur WebSocket n'est pas démarré (6.2).
 - « ⚠ **Streamer.bot désactivé** » : coche « Se connecter à Streamer.bot » dans `reglages.html`.
4. Chaque événement reçu s'y ajoute (ex. `Twitch.Follow` → `follow` · `Pseudo`), avec **les données brutes** reçues en dessous.
5. Une fois vérifié, **retire** `?journal=1` de l'adresse (sinon le panneau reste à l'écran pendant le live).

6.4 Événement par événement

Une fois Streamer.bot branché, **rien à régler par événement** : l'overlay les écoute tous. Les textes se changent dans `reglages.html` > **Alertes** (avec un aperçu).

Événement Twitch	Alerte
Follow	Nouveau témoin ! – va assister au carnage
Abonnement	Soutien moral officiel – croit encore en moi (courageux)
Réabonnement	Toujours là ?! – supporte ça depuis X mois
Abonnement offert	Cadeau empoisonné – offre un abonnement à...
Pluie d'abonnements	Pluie de cadeaux ! (<i>grande alerte</i>)
Bits	Pièces de consolation – envoie X bits pour me remonter le moral
Raid	Invasion !! – débarque avec X personnes (<i>grande alerte</i>)
Don	Fonds pour excuses – finance ma prochaine excuse
Objectif atteint	Objectif atteint ! – même moi je n'y croyais pas (<i>grande alerte</i>)

Abonnements, abonnements offerts et bits demandent une chaîne **affiliée** ou **partenaire**. Les alertes passent **une par une** (file d'attente) : rien n'est perdu pendant un raid.

6.5 Les dons (facultatif)

Twitch ne gère pas les dons en argent : ils passent par un service (StreamElements, Streamlabs, Ko-fi, Tipeee). Dans Streamer.bot, onglet **Integrations**, choisis ton service et suis ses indications (en général, coller une **clé** copiée depuis le tableau de bord du service, puis **Connect**).

Si tu ne reçois ni dons ni bits, décoche la case correspondante du bandeau : `reglages.html` > **Bandeau d'infos**.

6.6 L'objectif

`reglages.html` > **Objectif** : ce qu'on compte (**les follows** ou **les abonnements**), le nom, la cible et **ton nombre ACTUEL**. Le compteur avance à chaque follow (ou abonnement, une pluie d'abonnements comptant pour tous ses cadeaux) reçu **pendant que OBS est ouvert**, et s'en souvient d'un live à l'autre. De temps en temps, remets ton vrai nombre dans « Ton nombre ACTUEL ».

6.7 Tester les vraies alertes

- **Sans Twitch** : `?test=1` sur une page (fausses alertes toutes les 9 secondes, sans toucher au vrai compteur), ou `reglages.html` > **Tester**.
- **Pour de vrai** : demande à un ami (ou à un deuxième compte) de suivre la chaîne, OBS et Streamer.bot ouverts.

 **Pas encore vérifié sur un vrai live** : si une alerte montre « Quelqu'un » ou « ? », fais une capture d'écran du journal (6.3) pour faire corriger l'overlay.

6.8 Les sons des alertes

Chaque alerte a **son propre son**, pour savoir ce qui se passe à l'oreille, même en pleine partie :

Alerte	Le son
Follow	un « pop » et deux notes
Abonnement	un « pop », un sifflet qui monte et un « ding »
Réabonnement	deux « pop » et deux notes
Abonnement offert	un « pop » et des étincelles
Pluie d'abonnements	une rafale de « pop », une gamme et un « boing »
Bits	une pièce de jeu vidéo
Raid	une grosse éclaboussure, une gamme et un « boing »
Don	un « ta-daa » au kazoo
Objectif atteint	la gamme, le « boing » et un accord final

Les écouter : `reglages.html` > **Sons des alertes**, bouton ► à côté de chaque alerte.

Mettre ton propre son (un mp3, wav ou ogg, court de préférence) :

1. Copie ton fichier dans le dossier `sons/` de l'overlay, par exemple `sons/follow.mp3`.
2. `reglages.html` > **Sons des alertes** : dans la case de l'alerte, écris `sons/follow.mp3`. Clique ► pour vérifier.
3. **Enregistrer**, puis dans OBS : clic droit sur la source des alertes > **Actualiser**.

Écris `aucun` dans une case pour que cette alerte reste silencieuse ; vide la case pour revenir au son de l'overlay.

Le volume général et le bouton « son » sont dans `reglages.html` > **Alertes** ; dans OBS, le volume se règle aussi dans le mélangeur audio (case **Contrôler l'audio via OBS** de la source des alertes).

7. HABILLER LA CHAÎNE TWITCH

Tous les visuels sont dans `chaîne/` : ouvre `chaîne/kit.html` pour les voir. Les images prêtes à envoyer sont dans `chaîne/export/`.

Visuel	Fichier	Où l'envoyer sur Twitch
Photo de profil	profil.png (800 × 800)	Tableau de bord des créateurs > Paramètres > Chaîne > Marque > Photo de profil
Bannière de profil	banniere.png (1200 × 480)	... > Marque > Bannière de profil
Écran hors-ligne	hors-ligne.png (1920 × 1080)	... > Marque > Bannière du lecteur vidéo
Panneaux de bio	panneau-a-propos.png , panneau-planning.png , panneau-regles.png , panneau-materiel.png , panneau-soutenir.png (320 × 160)	Ta chaîne > onglet À propos > Modifier les panneaux > +
Emotes	emote-panique , lag , gg , oups , exclamation , coeur (-112, -56, -28)	Tableau de bord > Récompenses des spectateurs > Emotes (<i>affilié ou partenaire</i>)
Badges d'abonné	badge-mois-1 (jauge 20), -3 (40), -6 (60), -9 (80), -12 (étoile) (-72, -36, -18)	Tableau de bord > Récompenses des spectateurs > Badges d'abonné

Les badges montrent une **jauge de skill** qui se remplit avec l'ancienneté : 1 mois · Témoin, 3 mois · Complice, 6 mois · Soutien moral, 9 mois · Expert en excuses, 1 an · Légende du carnage.

Les panneaux, pas à pas :

1. Va sur ta chaîne, onglet **À propos**, active **Modifier les panneaux**.
2. + > **Ajouter un panneau texte ou image**.
3. **Image** : choisis le PNG du panneau. **Description** : écris le texte (qui tu es, le planning, les règles du chat...).
4. **Envoyer**, puis recommence pour les autres panneaux.

Changer les textes (slogan, texte hors-ligne, planning, titres des panneaux) : `reglages.html` > **Kit de chaîne Twitch**, puis refais les images dans PowerShell :

```
cd G:\Projets\Overlay\ambries
node outils/exporter-chaîne.mjs
```

Les noms des menus Twitch changent parfois un peu : si tu ne trouves pas un intitulé, cherche « Marque » ou « Récompenses des spectateurs » dans le tableau de bord.

8. TESTER SANS ÊTRE EN LIVE

Option	Effet
?test=1	faux messages de chat et fausses alertes qui défilent
?apercu=1	affiche les zones de la cam et du jeu
?journal=1	affiche le journal : connexion à Streamer.bot et derniers événements reçus (6.3)
?chat=0 · ?bandeau=0	retire le chat ou le bandeau intégré à la scène
?cam=0	(Jeu, Contenu) pas de webcam : le cadre disparaît et le chat s'agrandit
?cam=1200,700,420,236	(Jeu) webcam à une position perso : x, y, largeur, hauteur (pixels 1920 × 1080)
?minutes=0.5	(démarrage) compte à rebours de 30 s, pour voir la fin vite
?reinitialiser	remet le compteur de l'objectif à sa valeur de départ

Le mode test **ne modifie pas** le vrai compteur de l'objectif. Le plus simple : ouvre `index.html` (tout y tourne en mode test), ou `reglages.html` > **Tester**.

9. PERSONNALISER

Presque tout se règle dans `reglages.html` (2.1), sans toucher au code.

Je veux changer...	Où
Les textes, titres, messages, l'objectif	<code>reglages.html</code> (ou <code>config.js</code> à la main)
La devise (« 10 % skill · 90 % d'excuses »)	<code>reglages.html</code> > La chaîne
Les phrases du démarrage	<code>reglages.html</code> > Démarrage
Le vocabulaire des alertes (avec aperçu)	<code>reglages.html</code> > Alertes
Le son de chaque alerte, ou ton propre fichier	<code>reglages.html</code> > Sons des alertes (voir 6.8)
La durée, le volume ou le son des alertes	<code>reglages.html</code> > Alertes
Les bots masqués, la mémoire du chat	<code>reglages.html</code> > Le chat
Les cases du bandeau (dons, abonnés...)	<code>reglages.html</code> > Bandeau d'infos
Les visuels de la chaîne	<code>reglages.html</code> > Kit de chaîne Twitch (puis <code>node outils/exporter-chaîne.mjs</code> , voir « Les scripts »)
L'avatar	remplace <code>assets/avatar.png</code> (carré, 800 × 800 conseillé), puis refais vidéos et images
Le coin ou la position exacte de la webcam, pas de webcam, le chat ou le bandeau d'une scène	<code>reglages.html</code> > Options des scènes
Les couleurs, une ambiance (Halloween, Noël...)	<code>reglages.html</code> > Couleurs (voir ci-dessous)
Les couleurs d'origine, les polices	début de <code>css/theme.css</code>
Remettre l'objectif à zéro	<code>reglages.html</code> > Objectif > Ton nombre ACTUEL

Dans les textes des alertes, `{nom}`, `{montant}`, `{mois}`, `{nombre}` et `{destinataire}` sont remplacés automatiquement.

Changer d'ambiance (Halloween, Noël...) en un clic

1. Ouvre `reglages.html` > section **Couleurs**.
2. Clique 🎃 **Halloween** (ou 🎄 **Noël**), ou change une couleur à la main (le nuancier, ou un code comme `#FF7A1A`). 🔄 remet la couleur d'origine d'une seule couleur ; 🔄 **Couleurs d'origine** les remet toutes.
3. **Enregistrer**, puis actualise les sources dans OBS (ou laisse faire le script de la section 2) : toutes les scènes, sources et alertes prennent ces couleurs.

Créer ta propre ambiance (ex. Batman) et la garder

1. Dans `reglages.html` > **Couleurs**, règle les couleurs comme tu veux (nuancier ou code).
2. Sous **Mes ambiances**, tape un nom (ex. `Batman`) puis clique 💾 **Sauvegarder ces couleurs** : l'ambiance est écrite tout de suite dans `mes-reglages.js`, avec son nom et ses couleurs. Même nom qu'une ambiance existante = elle est remplacée (la page demande confirmation).
3. Elle apparaît ensuite en bouton : **un clic** remet toutes ses couleurs, puis **Enregistrer** pour que l'overlay les prenne. ✕ la supprime.

Sauvegarder une ambiance ne change pas les couleurs de l'overlay : seul **Enregistrer** le fait.

Les **vidéos de transition** et les **images du kit de chaîne** sont déjà fabriquées : pour qu'elles prennent les nouvelles couleurs, refais-les avec les scripts ci-dessous : `node outils/generer-transitions.mjs` puis `node outils/exporter-chaîne.mjs` (et quand tu reviens aux couleurs d'origine, pareil).

Les scripts du dossier `outils/`

Ces scripts **refont les fichiers « fabriqués »** : vidéos de transition, images du kit, PDF... On ne s'en sert qu'après un changement (couleurs, textes du kit, tuto modifié). Tout le reste (réglages, textes, alertes) se fait dans `reglages.html` , sans script.

Une seule fois : installer Node.js (le programme qui lance les scripts `.mjs`)

1. Va sur le site officiel **nodejs.org**, télécharge la version **LTS** et installe-la (tout laisser par défaut, *Suivant* jusqu'au bout). Ou, dans PowerShell : `winget install OpenJS.NodeJS.LTS`
2. Pour **refaire les vidéos de transition**, il faut aussi **ffmpeg** : dans PowerShell, `winget install Gyan.FFmpeg` .
3. Ferme puis rouvre PowerShell.

Lancer un script

1. Ouvre le dossier `ambries` dans l'Explorateur Windows.
2. Clic droit dans un espace vide du dossier > **Ouvrir dans le Terminal** (ou tape `powershell` dans la barre d'adresse du dossier, puis **Entrée**).
3. Tape la commande du tableau, puis **Entrée**. Le script dit ce qu'il fait, puis rend la main.

Script	À quoi il sert	Quand	Commande
<code>actualiser-obs.lua</code>	bouton « Actualiser toutes les sources Navigateur » dans OBS, bouton « Placer les webcams » (et remplacement automatique), et actualisation automatique quand tes réglages changent	une fois, à installer dans OBS (section 2)	<i>pas de commande</i> : OBS > Outils > Scripts > +
<code>generer-transitions.mjs</code>	refait les vidéos <code>transitions/videos/*.webm</code> (Stinger)	après un changement de couleurs	<code>node outils/generer-transitions.mjs</code> (<i>ffmpeg nécessaire</i>)
<code>exporter-chaine.mjs</code>	refait les images <code>chaine/export/*.png</code> (profil, bannière, panneaux, emotes, badges)	après un changement de couleurs ou des textes du kit	<code>node outils/exporter-chaine.mjs</code>
<code>generer-pdf.mjs</code>	refait <code>TUTO.pdf</code> et <code>CONCEPT.pdf</code> depuis les <code>.md</code>	après une modification de <code>TUTO.md</code> ou <code>CONCEPT.md</code>	<code>node outils/generer-pdf.mjs</code>
<code>capture.r.mjs</code>	fait une capture PNG d'une page (pour vérifier une animation, ou l'envoyer)	pour vérifier	<code>node outils/capturer.mjs "scenes/jeu.html?test=1"</code>
<code>animer.mjs</code>	refait les captures animées <code>captures/*.webp</code> (écrans, scènes, alertes, transitions) à partager ; galerie : <code>captures/index.html</code>	après un changement, pour montrer le résultat	<code>node outils/animer.mjs captures/liste.json</code> (<i>ffmpeg nécessaire</i>)
<code>polices-locales.mjs</code>	copie dans <code>assets/polices/</code> les polices du thème (pour ne plus dépendre d'internet)	seulement si on change de police (déjà fait)	<code>node outils/polices-locales.mjs</code> (<i>internet nécessaire</i>)

Les scripts se servent de **Microsoft Edge** en coulisses (déjà installé avec Windows) : rien d'autre à installer.

10. DÉPANNAGE

Mes réglages ont disparu après une mise à jour de l'overlay

→ Ton `mes-reglages.js` a peut-être été remplacé ou supprimé : il ne doit **pas** faire partie des fichiers que tu copies. S'il te reste une copie, remets-la dans le dossier de l'overlay.

→ Si tes réglages étaient encore dans l'ancien `config.js` (avant `mes-reglages.js`) : `reglages.html` >

 **Reprendre les réglages d'un ancien config.js** > choisis une copie de cet ancien fichier, puis **Enregistrer** (section 2.1).

Rien ne s'affiche / page blanche

→ Vérifie `1920 × 1080` sur la source, puis clic droit > **Actualiser**. Si tu viens de modifier `config.js`, cherche un guillemet ou une virgule manquant.

Les polices ne sont pas les bonnes

→ Elles sont dans `assets/polices/` (pas besoin d'internet) : vérifie que le dossier est bien là, à côté de `css/`, puis actualise la source.

Le chat n'affiche rien

→ Vérifie l'identifiant Twitch dans `reglages.html` > **La chaîne** (l'identifiant exact, en minuscules). Le chat n'affiche que les messages envoyés **après** l'ouverture de la page, et il a besoin d'internet.

Les alertes ne s'affichent pas

→ Streamer.bot est-il lancé, serveur WebSocket **démarré** (port `8080`), compte Broadcaster connecté ? Regarde le journal (6.3, `?journal=1`) : il dit ce qui coince.

Les réglages ne changent rien dans OBS

→ Après **Enregistrer** dans `reglages.html`, il faut actualiser les sources : clic droit > **Actualiser**, ou le script `actualiser-obs.lua` (section 2) qui le fait tout seul.

On n'entend pas le son des alertes

→ Coche **Contrôler l'audio via OBS** sur la source d'alertes et vérifie son volume dans le mélangeur.

Une alerte s'affiche mal (pseudo manquant, « ? »...)

→ OBS n'a pas de console (F12) : ajoute `?journal=1` à l'adresse de la source d'alertes (décoche *Fichier local*, colle l'adresse de la page suivie de `?journal=1`). Un panneau affiche la connexion à Streamer.bot («  Connecté ») et chaque événement reçu avec ses données brutes : fais-en une capture d'écran pour faire corriger l'overlay, puis retire `?journal=1`.

La cam ne tombe pas pile dans le cadre

→ Le plus simple : OBS > **Outils** > **Scripts** > `actualiser-obs.lua` > **Placer les webcams sur toutes les scènes** (le nom de ta webcam doit contenir « cam »). Sinon :

→ Refais le geste B, et vérifie que la webcam est bien **sous** l'overlay.

Le compte à rebours ou le chrono de pause ne repart pas de zéro

→ Coche **Actualiser le navigateur quand la scène devient active** sur la source.

Une ancienne image s'affiche encore (après une modification de `config.js`, de l'avatar ou des couleurs)

→ Dans OBS : clic droit sur la source > **Propriétés** > **Actualiser le cache de la page actuelle**. Dans un navigateur : **Ctrl + F5**.

Bon live, et bon courage pour les excuses ! 